

Engineering Command Compendium

Useful commands for Developers, Cloud Architects, DevOps, Platform and SRE Engineers

Contents

- 1. File and Directory Management** — [file-directory-management-commands.md](#)
- 2. File Viewing and Editing** — [file-viewing-editing-commands.md](#)
- 3. Process Management** — [process-management-commands.md](#)
- 4. Disk Management** — [disk-management-commands.md](#)
- 5. Networking** — [networking-commands.md](#)
- 6. User and Group Management** — [user-group-management-commands.md](#)
- 7. System Information and Monitoring** — [system-information-monitoring-commands.md](#)
- 8. Archiving and Compression** — [archiving-compression-commands.md](#)
- 9. Package Management (Depends on Distribution)** — [package-management-commands.md](#)
- 9A. System Services and Daemon Management** — [system-services-daemon-management-commands.md](#)
- 10. Scheduling Tasks** — [scheduling-tasks-commands.md](#)
- 11. File Permissions and Security** — [file-permissions-security-commands.md](#)
- 12. System Backup and Restore** — [system-backup-restore-commands.md](#)
- 13. System Diagnostics and Troubleshooting** — [system-diagnostics-troubleshooting-commands.md](#)
- 14. Networking & Remote Management** — [networking-remote-management-commands.md](#)
- 15. Text Processing Utilities** — [text-processing-utilities-commands.md](#)
- 16. System Shutdown and Reboot** — [system-shutdown-reboot-commands.md](#)
- 17. File System Mounting and Management** — [filesystem-mounting-management-commands.md](#)
- 18. Filesystem Permissions and Security** — [filesystem-permissions-security-commands.md](#)
- 19. Containerization and Orchestration** — [containerization-orchestration-commands.md](#)
- 20. Automation and Configuration Management** — [automation-configuration-management-commands.md](#)
- 21. CI/CD Tools and Commands** — [cicd-tools-commands.md](#)
- 22. Cloud Services** — [cloud-services-cli-commands.md](#)
- 23. Logging and Monitoring** — [logging-monitoring-commands.md](#)
- 24. Git Version Control** — [git-version-control-commands.md](#)
- 25. Shell and Environment** — [shell-environment-commands.md](#)
- 26. JSON, YAML and Structured Data** — [json-yaml-structured-data-commands.md](#)
- 27. Security, Cryptography and Certificates** — [security-crypto-certificates-commands.md](#)
- 28. Database CLI** — [database-cli-commands.md](#)
- 29. GitHub CLI** — [github-cli-commands.md](#)
- 30. Python and Virtual Environments** — [python-virtual-environment-commands.md](#)
- 31. Node.js and npm** — [nodejs-npm-commands.md](#)
- 32. Anthropic Claude / Claude Code** — [anthropic-claude-commands.md](#)

How to Read This Compendium

- Commands assume Linux/POSIX semantics unless the category explicitly targets another tool or platform.
- Placeholders such as HOST, USER, FILE, PACKAGE, CLUSTER and RG must be replaced with values appropriate to your environment.
- Always confirm your target system, cloud account, Kubernetes context and current directory before running destructive commands.

1. File and Directory Management

Markdown file: commands/file-directory-management-commands.md

Command / Example	Purpose	Notes
ls	List directory contents.	
cd	Change the current directory.	
pwd	Print the current working directory.	
cp	Copy files and directories.	
mv	Move or rename files and directories.	
rm	Remove files or directories.	Destructive; verify paths before using recursive/force options.
mkdir	Create directories.	
rmdir	Remove empty directories.	
touch	Create an empty file or update file timestamps.	
find	Search files and directories in a hierarchy.	
locate	Find files by name using an indexed database.	
tree	Display directories in a tree-like format.	
chmod	Change file permissions.	
chown	Change file owner and group.	
chgrp	Change group ownership.	
stat	Display file or filesystem status.	
ls -lah	List all files with human-readable sizes and details.	
cp -a SOURCE DEST	Archive-style copy preserving permissions, timestamps and symlinks.	
mkdir -p path/to/dir	Create nested directories without failing when parents already exist.	
rm -rf PATH	Recursively and forcibly remove a path.	High-risk destructive command.
find . -type f -name "*.log"	Find .log files below the current directory.	
find . -type f -mtime +30	Find files modified more than 30 days ago.	
ln -s TARGET LINK	Create a symbolic link.	
readlink -f PATH	Resolve a path or symbolic link to its canonical path.	
realpath PATH	Print an absolute normalized path.	
basename PATH	Return the filename portion of a path.	
dirname PATH	Return the directory portion of a path.	
file PATH	Identify a file type from its contents.	
mktemp	Create a secure temporary file or directory.	

2. File Viewing and Editing

Markdown file: [commands/file-viewing-editing-commands.md](#)

Command / Example	Purpose	Notes
cat	Concatenate and display file content.	
tac	Display file content in reverse line order.	
more	View file content page by page.	
less	View file content interactively with scrolling and search.	
head	Output the first part of a file.	
tail	Output the last part of a file.	
nano	Terminal-based text editor.	
vim / vi	Advanced terminal text editors.	
emacs	Text editor.	
grep	Search text using patterns.	
sed	Stream editor for filtering and transforming text.	
awk	Pattern scanning and text-processing language.	
cut	Remove or select sections from each line.	
sort	Sort lines of text files.	
uniq	Report or omit repeated adjacent lines.	
tail -f FILE	Follow a file as new lines are appended, useful for logs.	
head -n 20 FILE	Show the first 20 lines.	
less +F FILE	Open a file and follow appended output interactively.	
nl -ba FILE	Display a file with line numbers, including blank lines.	
strings BINARY	Extract printable strings from a binary file.	
diff -u OLD NEW	Show a unified diff between two files.	
cmp FILE1 FILE2	Compare two files byte by byte.	
patch -p1 < change.patch	Apply a patch file.	

3. Process Management

Markdown file: [commands/process-management-commands.md](#)

Command / Example	Purpose	Notes
ps	Report a snapshot of current processes.	
top	Display and monitor Linux tasks.	

Command / Example	Purpose	Notes
htop	Interactive process viewer (when installed).	
kill PID	Send a signal to a process, commonly to request termination.	
killall NAME	Send a signal to processes by name.	
bg	Resume a suspended job in the background.	
fg	Bring a job to the foreground.	
jobs	List active jobs in the current shell.	
nice -n 10 COMMAND	Run a program with a modified scheduling priority.	
renice 10 -p PID	Alter the priority of a running process.	
uptime	Show system uptime and load averages.	
time COMMAND	Measure command runtime and resource use.	
ps aux --sort=-%mem head	Show the top memory-consuming processes.	
ps aux --sort=-%cpu head	Show the top CPU-consuming processes.	
pgrep -af PATTERN	Find processes by pattern and print full command lines.	
pkill -TERM PATTERN	Send a signal to matching processes.	
pstree -p	Display processes as a tree with PIDs.	
nohup COMMAND >app.log 2>&1 &	Run a command so it can continue after logout.	
disown %JOB	Detach a shell job from the current shell.	
timeout 30s COMMAND	Stop a command if it runs longer than the specified duration.	
kill -STOP PID	Suspend a process.	
kill -CONT PID	Resume a suspended process.	
kill -9 PID	Force-kill a process.	Use only when graceful termination fails.

4. Disk Management

Markdown file: [commands/disk-management-commands.md](#)

Command / Example	Purpose	Notes
df	Report filesystem disk-space usage.	
du	Estimate file or directory space usage.	
fdisk	Manipulate MBR/GPT partition tables on Linux.	

Command / Example	Purpose	Notes
lsblk	List information about block devices.	
mount	Mount a filesystem.	
umount	Unmount a filesystem.	
parted	Create and manipulate disk partitions.	
mkfs	Create a filesystem.	
fsck	Check and repair a filesystem.	
blkid	Locate or print block-device attributes.	
df -hT	Show filesystem usage in human-readable form with filesystem type.	
du -sh PATH	Show total size of a path.	
du -h --max-depth=1 PATH sort -h	Show first-level directory sizes, sorted.	
lsblk -f	Show block devices, filesystem types, labels and UUIDs.	
fdisk -l	List partition tables.	
parted -l	List disks and partitions known to parted.	
findmnt	List mounted filesystems in a structured tree.	
swapon --show	Show active swap devices/files.	
sync	Flush pending filesystem writes to storage.	

5. Networking

Markdown file: [commands/networking-commands.md](#)

Command / Example	Purpose	Notes
ifconfig	Configure or display network interfaces.	Legacy on many distributions; prefer ip.
ip	Show or manipulate routing, devices, interfaces and tunnels.	
ping HOST	Send ICMP echo requests to a host.	
netstat	Display network statistics and connections.	Legacy on many distributions; prefer ss.
ss	Display socket statistics.	
traceroute HOST	Trace the route packets take to a host.	
nslookup NAME	Query DNS servers interactively.	
dig NAME	Perform DNS lookups.	
wget URL	Download content non-interactively.	
curl URL	Transfer data using URLs.	

Command / Example	Purpose	Notes
scp SOURCE DEST	Copy files securely over SSH.	
ssh HOST	Open a secure remote shell.	
ftp HOST	Connect using FTP.	Plain FTP is insecure; prefer SFTP/SCP/HTTPS.
ip addr	Show IP addresses assigned to interfaces.	
ip link	Show or manipulate network interfaces.	
ip route	Show or manipulate the routing table.	
ip neigh	Show or manage ARP/neighbor entries.	
ss -tulpn	Show listening TCP/UDP sockets with process information.	
hostname -I	Print host IP addresses.	
resolvectl status	Show systemd-resolved DNS status when available.	
dig +short NAME	Return concise DNS answers.	
dig @SERVER NAME	Query a specific DNS server.	
curl -I URL	Fetch HTTP response headers only.	
curl -v URL	Show verbose connection and protocol details.	
nc -vz HOST PORT	Test whether a TCP port is reachable.	
mtr HOST	Continuously combine ping and traceroute diagnostics (when installed).	
tracepath HOST	Trace network path without requiring raw-socket privileges on many systems.	
ethtool INTERFACE	Show or configure Ethernet adapter settings.	
nmcli device status	Show NetworkManager device status.	

6. User and Group Management

Markdown file: [commands/user-group-management-commands.md](#)

Command / Example	Purpose	Notes
useradd USER	Add a user account.	
usermod	Modify a user account.	
userdel USER	Delete a user account.	
groupadd GROUP	Add a group.	
groupdel GROUP	Delete a group.	
passwd USER	Change a user password.	
chage	Change password expiry information.	
whoami	Print the current effective username.	

Command / Example	Purpose	Notes
who	Show logged-in users.	
w	Show logged-in users and what they are doing.	
id	Display user and group IDs.	
groups USER	Show a user's groups.	
useradd -m -s /bin/bash USER	Create a user with a home directory and Bash shell.	
usermod -aG GROUP USER	Add a user to an additional group without replacing existing memberships.	
groupmod -n NEW OLD	Rename a group.	
getent passwd USER	Query the configured account databases for a user.	
getent group GROUP	Query group information.	
last	Show login history.	
lastlog	Show most recent login for users.	
su - USER	Switch to another user with a login shell.	
sudo -u USER COMMAND	Run a command as another user.	

7. System Information and Monitoring

Markdown file: commands/system-information-monitoring-commands.md

Command / Example	Purpose	Notes
uname	Print system information.	
hostname	Show or set the system's hostname.	
uptime	Show uptime and load averages.	
dmesg	Display kernel ring-buffer messages.	
free	Display memory usage.	
top	Display Linux tasks.	
vmstat	Report virtual-memory statistics.	
lscpu	Display CPU architecture information.	
lsusb	List USB devices.	
lspci	List PCI devices.	
lshw	List hardware configuration.	
uname -a	Show kernel, hostname, architecture and other system details.	
cat /etc/os-release	Show Linux distribution and version information.	
hostnamectl	Show host, kernel and OS information on systemd systems.	
free -h	Show memory and swap usage in human-readable units.	

Command / Example	Purpose	Notes
<code>vmstat 1</code>	Sample CPU, memory, process and I/O statistics every second.	
<code>iostat -xz 1</code>	Show extended CPU and device I/O statistics every second (sysstat package).	
<code>sar -u 1 5</code>	Sample CPU statistics using sysstat.	
<code>sensors</code>	Display hardware temperature and sensor readings (lm-sensors).	
<code>lsmem</code>	List memory ranges and memory-block state.	

8. Archiving and Compression

Markdown file: [commands/archiving-compression-commands.md](#)

Command / Example	Purpose	Notes
<code>tar -czf archive.tar.gz /path/to/directory</code>	Create a gzip-compressed tar archive.	
<code>tar -xzf archive.tar.gz</code>	Extract a gzip-compressed tar archive.	
<code>tar -cf archive.tar /path/to/directory</code>	Create an uncompressed tar archive.	
<code>tar -xf archive.tar</code>	Extract a tar archive.	
<code>zip</code>	Package and compress files into a ZIP archive.	
<code>unzip</code>	Extract a ZIP archive.	
<code>gzip</code>	Compress files using gzip.	
<code>gunzip</code>	Decompress gzip files.	
<code>bzip2</code>	Compress files using bzip2.	
<code>bunzip2</code>	Decompress bzip2 files.	
<code>xz</code>	Compress files using xz.	
<code>unxz</code>	Decompress xz files.	
<code>tar -caf archive.tar.xz PATH</code>	Create an archive and choose compression automatically from the suffix.	
<code>tar -taf archive.tar.gz</code>	List archive contents without extracting.	
<code>zip -r archive.zip DIRECTORY</code>	Recursively create a ZIP archive.	
<code>unzip -l archive.zip</code>	List ZIP contents without extracting.	
<code>zstd FILE</code>	Compress a file using Zstandard (when installed).	
<code>unzstd FILE.zst</code>	Decompress a Zstandard file.	

Command / Example	Purpose	Notes
<code>7z a archive.7z PATH</code>	Create a 7z archive (7-Zip/p7zip when installed).	
<code>7z x archive.7z</code>	Extract a 7z archive.	

9. Package Management (Depends on Distribution)

Markdown file: `commands/package-management-commands.md`

Command / Example	Purpose	Notes
<code>apt-get install PACKAGE</code>	Install a package on Debian/Ubuntu.	
<code>apt-get update</code>	Update APT package indexes.	
<code>apt-get upgrade</code>	Upgrade installed packages.	
<code>apt-get remove PACKAGE</code>	Remove a package.	
<code>apt-cache search PACKAGE</code>	Search APT package metadata.	
<code>apt-cache show PACKAGE</code>	Show APT package details.	
<code>yum install PACKAGE</code>	Install an RPM package on yum-based systems.	
<code>yum update</code>	Update installed packages on yum-based systems.	
<code>yum remove PACKAGE</code>	Remove a package using yum.	
<code>dnf install PACKAGE</code>	Install a package on modern Fedora/RHEL-family systems.	
<code>dnf update</code>	Update installed packages using dnf.	
<code>dnf remove PACKAGE</code>	Remove a package using dnf.	
<code>rpm -i PACKAGE.rpm</code>	Install an RPM package directly.	
<code>rpm -e PACKAGE</code>	Remove an RPM package.	
<code>dpkg -i PACKAGE.deb</code>	Install a Debian package directly.	
<code>dpkg -r PACKAGE</code>	Remove a Debian package.	
<code>apt install PACKAGE</code>	Preferred interactive APT install command.	
<code>apt update && apt upgrade</code>	Refresh indexes and upgrade packages.	
<code>apt autoremove</code>	Remove automatically installed packages no longer needed.	
<code>apt search PATTERN</code>	Search packages.	
<code>dpkg -l</code>	List installed Debian packages.	
<code>dpkg -S PATH</code>	Find which installed package owns a path.	
<code>dnf search PATTERN</code>	Search dnf repositories.	
<code>dnf info PACKAGE</code>	Show package information.	
<code>rpm -qa</code>	List installed RPM packages.	

Command / Example	Purpose	Notes
<code>rpm -qf PATH</code>	Find which installed RPM owns a file.	
<code>pacman -S PACKAGE</code>	Install a package on Arch Linux.	
<code>pacman -Syu</code>	Refresh package database and fully upgrade Arch Linux.	
<code>zypper install PACKAGE</code>	Install a package on openSUSE/SLES.	
<code>zypper update</code>	Update packages on openSUSE/SLES.	
<code>brew install PACKAGE</code>	Install a package using Homebrew on macOS/Linux.	
<code>brew update && brew upgrade</code>	Update Homebrew metadata and upgrade installed formulae.	
<code>brew services list</code>	List services managed by Homebrew.	

9A. System Services and Daemon Management

Markdown file: `commands/system-services-daemon-management-commands.md`

Command / Example	Purpose	Notes
<code>systemctl start SERVICE</code>	Start a systemd service.	
<code>systemctl stop SERVICE</code>	Stop a systemd service.	
<code>systemctl restart SERVICE</code>	Restart a systemd service.	
<code>systemctl enable SERVICE</code>	Enable a service at boot.	
<code>systemctl disable SERVICE</code>	Disable a service at boot.	
<code>systemctl status SERVICE</code>	Check service status.	
<code>service SERVICE start</code>	Start a service on SysV/init-compatible systems.	
<code>service SERVICE stop</code>	Stop a service on SysV/init-compatible systems.	
<code>service SERVICE restart</code>	Restart a service on SysV/init-compatible systems.	
<code>service SERVICE status</code>	Check legacy service status.	
<code>systemctl enable --now SERVICE</code>	Enable a service at boot and start it immediately.	
<code>systemctl disable --now SERVICE</code>	Disable a service and stop it immediately.	
<code>systemctl reload SERVICE</code>	Reload service configuration without a full restart when supported.	
<code>systemctl list-units --type=service --state=running</code>	List running services.	
<code>systemctl --failed</code>	List failed systemd units.	
<code>systemctl cat SERVICE</code>	Show the effective unit file and drop-ins.	
<code>systemctl edit SERVICE</code>	Create or edit a systemd override drop-in.	

Command / Example	Purpose	Notes
<code>systemctl daemon-reload</code>	Reload systemd manager configuration after unit-file changes.	
<code>journalctl -u SERVICE -f</code>	Follow logs for a systemd service.	

10. Scheduling Tasks

Markdown file: [commands/scheduling-tasks-commands.md](#)

Command / Example	Purpose	Notes
<code>crontab -e</code>	Edit the current user's cron jobs.	
<code>crontab -l</code>	List the current user's cron jobs.	
<code>crontab -r</code>	Remove the current user's cron jobs.	Destructive; removes the current crontab.
<code>at 09:00</code>	Schedule commands to run at 09:00.	
<code>batch</code>	Run queued commands when system load is low.	
<code>sleep 5s</code>	Delay for five seconds.	
<code>crontab -u USER -l</code>	List another user's crontab (requires appropriate privileges).	
<code>atq</code>	List pending at jobs.	
<code>atrm JOB_ID</code>	Remove a pending at job.	
<code>systemctl list-timers -all</code>	List systemd timers and their next/last run times.	
<code>systemd-run --on-active=10m COMMAND</code>	Create a transient systemd timer that runs once after ten minutes.	
<code>watch -n 5 COMMAND</code>	Run a command repeatedly every five seconds in an interactive terminal.	

11. File Permissions and Security

Markdown file: [commands/file-permissions-security-commands.md](#)

Command / Example	Purpose	Notes
<code>chmod</code>	Change file permissions.	
<code>chown</code>	Change file owner and group.	
<code>chgrp</code>	Change group ownership of a file.	
<code>umask</code>	Set default permission mask for newly created files.	
<code>setfacl</code>	Set POSIX access-control lists (ACLs).	
<code>getfacl</code>	Read POSIX access-control lists.	
<code>sudo</code>	Execute a command as another user, usually root.	
<code>visudo</code>	Edit the sudoers policy safely.	

Command / Example	Purpose	Notes
passwd	Change a user's password.	
/etc/sudoers	Sudo policy configuration file.	The screenshot labels “sudoers” as a command; it is a configuration file. Edit with visudo.
gpasswd	Administer group membership/passwords.	
ss	Display socket statistics.	The screenshot places ss in this section; it is primarily a networking diagnostic tool.
chmod 640 FILE	Set owner read/write, group read, others none.	
chmod u+x SCRIPT	Add execute permission for the owner.	
chown -R USER:GROUP PATH	Recursively change ownership.	
umask 027	Default to permissions that deny access to others.	
setfacl -m u:USER:rw FILE	Grant a named user read/write ACL permissions.	
getfacl FILE	Display ACL entries.	
sudo -l	Show commands the current user may run with sudo.	
visudo -c	Validate sudoers syntax.	
lsattr FILE	List extended filesystem attributes.	
chattr +i FILE	Mark a supported filesystem file immutable.	Requires care; remove with chattr -i before modification.
getcap -r / 2>/dev/null	List file capabilities recursively.	
setcap cap_net_bind_service=+ep BINARY	Grant a binary permission to bind low-numbered ports without full root privileges.	Use least privilege and review security implications.

12. System Backup and Restore

Markdown file: [commands/system-backup-restore-commands.md](#)

Command / Example	Purpose	Notes
rsync -avz SOURCE/ DESTINATION/	Synchronize files/directories with archive mode and compression.	
rsync -avz -e ssh SOURCE/ USER@HOST:/DEST/	Synchronize files over SSH.	
cpio	Copy files to and from cpio archives.	
dd	Low-level block/file copying.	

Command / Example	Purpose	Notes
<code>dd if=/dev/sda of=/path/to/backup.img</code>	Create a raw image backup of a disk/partition.	High-risk; verify source and destination carefully.
<code>dd if=/path/to/backup.img of=/dev/sda</code>	Restore a raw image to a disk/partition.	Destructive; overwrites the destination.
<code>rsync -aHAX --delete SOURCE/ DEST/</code>	Mirror a Linux tree while preserving hard links, ACLs and xattrs.	--delete removes destination files absent from source; test first.
<code>rsync -avn --delete SOURCE/ DEST/</code>	Dry-run a mirror operation before using --delete.	
<code>dd if=SOURCE of=DEST bs=64K status=progress conv=fsync</code>	Copy blocks with progress and flush data.	
<code>sha256sum BACKUP_FILE</code>	Generate an integrity checksum for a backup.	
<code>sha256sum -c checksums.sha256</code>	Verify checksums.	
<code>tar -czf backup.tar.gz PATH</code>	Create a compressed file-level backup.	
<code>cp -a SOURCE DEST</code>	Create a local copy preserving metadata.	
<code>sync</code>	Flush pending writes before detaching storage.	

13. System Diagnostics and Troubleshooting

Markdown file: `commands/system-diagnostics-troubleshooting-commands.md`

Command / Example	Purpose	Notes
<code>dmesg</code>	Print kernel ring-buffer messages.	
<code>journalctl</code>	Query and view systemd's journal.	
<code>strace COMMAND</code>	Trace a command's system calls and signals.	
<code>lsof</code>	List open files.	
<code>lsof FILE</code>	Show processes using a specific file.	
<code>vmstat</code>	Report virtual-memory statistics.	
<code>iostat</code>	Report CPU and I/O statistics.	
<code>mpstat</code>	Report CPU usage statistics.	
<code>pidstat</code>	Report statistics by process.	
<code>free</code>	Display memory usage.	
<code>uptime</code>	Show uptime and load averages.	
<code>watch -n 1 free</code>	Watch memory usage every second.	
<code>lshw</code>	List hardware configuration.	
<code>htop</code>	Interactive process viewer.	

Command / Example	Purpose	Notes
netstat	Display network statistics.	Legacy; prefer ss.
ss	Display socket statistics.	
journalctl -xe	Show recent journal messages with explanatory context.	
journalctl -u SERVICE -since "1 hour ago"	Show recent logs for a service.	
dmesg -T tail -100	Show recent kernel messages with human-readable timestamps.	
systemctl --failed	List failed systemd units.	
lsof -i :PORT	Show processes using a network port.	
fuser -v PATH	Show processes using a file, mount point or socket.	
strace -f -p PID	Attach to a process and follow system calls across child processes.	
pidstat -p PID 1	Sample resource usage for a specific process every second.	
iostat -xz 1	Inspect extended disk latency, queue and utilization metrics.	
vmstat 1	Inspect CPU run queue, memory, swap and I/O in real time.	

14. Networking & Remote Management

Markdown file: commands/networking-remote-management-commands.md

Command / Example	Purpose	Notes
ifconfig	Configure network interfaces.	Legacy; prefer ip.
ip addr	Show IP addresses.	
ip link	Show or manipulate network interfaces.	
ip route	Show or manipulate routing tables.	
ss	Display socket statistics.	
nmap	Network exploration and security-auditing tool.	
telnet HOST PORT	Connect using TELNET.	Plain telnet is insecure; commonly used only for simple connectivity tests.
nc -l -p 1234	Listen on TCP port 1234 using netcat.	
nc HOST PORT	Connect to a host and port using netcat.	
iptables	Administer legacy IPv4 packet filtering and NAT rules.	

Command / Example	Purpose	Notes
firewalld	Firewall management service used by several RPM-based distributions.	
ufw enable	Enable UFW firewall.	
ufw allow PORT	Allow traffic to a port using UFW.	
tcpdump	Capture and analyze packets from the command line.	
curl	Transfer data using HTTP and other protocols.	
wget	Download content over HTTP/HTTPS/FTP.	
scp FILE USER@HOST:/PATH/	Copy a file to a remote server over SSH.	
rsync -avz LOCAL/ HOST:/REMOTE/	Synchronize directories to a remote host.	
ssh -i KEY USER@HOST	Connect using a specific SSH private key.	
ssh -J BASTION TARGET	Connect to a target through a jump host.	
ssh -L 8080:REMOTE:80 USER@HOST	Create a local SSH port forward.	
sftp USER@HOST	Open an interactive secure file-transfer session.	
rsync -a --info=progress2 -e ssh SOURCE/ USER@HOST:/DEST/	Synchronize with aggregate progress over SSH.	
nft list ruleset	Display nftables firewall rules.	
firewall-cmd --list-all	Show firewalld configuration for the active zone.	
ufw status verbose	Show UFW rules and status.	
tcpdump -i any port 443	Capture traffic to/from TCP/UDP port 443 on all interfaces.	
openssl s_client -connect HOST:443 -servername HOST	Inspect a TLS connection and certificate chain.	

15. Text Processing Utilities

Markdown file: commands/text-processing-utilities-commands.md

Command / Example	Purpose	Notes
<code>grep 'pattern' file.txt</code>	Search a file for a pattern.	
<code>grep -r 'pattern' /dir/</code>	Recursively search files for a pattern.	
<code>sed 's/old/new/g' file.txt</code>	Replace all matches of old with new in output.	
<code>awk '{print \$1}' file.txt</code>	Print the first whitespace-delimited field from each line.	
<code>cut -d ':' -f 1 /etc/passwd</code>	Print the first colon-delimited field of each line.	
<code>sort file.txt</code>	Sort lines in ascending order.	
<code>sort file.txt uniq</code>	Sort and remove adjacent duplicate lines.	
<code>tee</code>	Write standard input to standard output and files.	
<code>echo "text" tee file.txt</code>	Write text to a file while displaying it.	
<code>echo 'hello' tr 'a-z' 'A-Z'</code>	Translate lowercase letters to uppercase.	
<code>paste file1.txt file2.txt</code>	Merge corresponding lines side by side.	
<code>wc -l file.txt</code>	Count lines.	
<code>wc -w file.txt</code>	Count words.	
<code>grep -nEi "ERROR WARN" FILE</code>	Search case-insensitively with line numbers and extended regex.	
<code>grep -v PATTERN FILE</code>	Exclude matching lines.	
<code>rg PATTERN PATH</code>	Fast recursive search with ripgrep (when installed).	
<code>sed -n '10,20p' FILE</code>	Print only lines 10 through 20.	
<code>awk -F, '{print \$1,\$3}' file.csv</code>	Print selected comma-separated fields.	
<code>sort FILE uniq -c sort -nr</code>	Count repeated lines and sort by frequency.	
<code>comm -12 <(sort A) <(sort B)</code>	Print lines common to two sorted inputs in Bash.	
<code>join FILE1 FILE2</code>	Join sorted text files on a common field.	
<code>xargs -n1 COMMAND < input.txt</code>	Build command invocations from standard input.	
<code>split -l 1000 FILE chunk-</code>	Split a file into chunks of 1,000 lines.	
<code>column -t FILE</code>	Align whitespace-separated data into columns.	

16. System Shutdown and Reboot

Markdown file: [commands/system-shutdown-reboot-commands.md](#)

Command / Example	Purpose	Notes
<code>shutdown -h now</code>	Shut down immediately.	
<code>shutdown -r now</code>	Reboot immediately.	
<code>shutdown -h +10</code>	Schedule shutdown in ten minutes.	
<code>reboot</code>	Reboot the system.	
<code>halt</code>	Halt the system.	
<code>poweroff</code>	Power off the system.	
<code>init 0</code>	Shutdown using legacy SysV init runlevel.	Legacy on modern systemd distributions.
<code>init 6</code>	Reboot using legacy SysV init runlevel.	Legacy on modern systemd distributions.
<code>shutdown -c</code>	Cancel a scheduled shutdown.	
<code>systemctl reboot</code>	Reboot a systemd host.	
<code>systemctl poweroff</code>	Power off a systemd host.	
<code>systemctl suspend</code>	Suspend a systemd host when supported.	

17. File System Mounting and Management

Markdown file: [commands/filesystem-mounting-management-commands.md](#)

Command / Example	Purpose	Notes
<code>mount /dev/sda1 /mnt</code>	Mount a partition at /mnt.	
<code>umount /mnt</code>	Unmount the filesystem mounted at /mnt.	
<code>/etc/fstab</code>	Persistent filesystem mount configuration file.	The screenshot labels “fstab” as a command; it is a configuration file.
<code>blkid</code>	Display block-device attributes.	
<code>fsck /dev/sda1</code>	Check and repair /dev/sda1.	Do not run on a mounted writable filesystem unless the filesystem documentation explicitly permits it.
<code>findmnt</code>	Show the current mount tree.	
<code>findmnt /mnt</code>	Show details for a specific mount point.	
<code>mount -a</code>	Mount filesystems configured in /etc/fstab that are eligible for automatic mounting.	

Command / Example	Purpose	Notes
<code>mount -o remount,rw MOUNTPOINT</code>	Remount a filesystem read-write.	
<code>mount -o ro DEVICE MOUNTPOINT</code>	Mount a filesystem read-only.	
<code>lsblk -f</code>	Show filesystems, labels, UUIDs and mount points.	
<code>blkid DEVICE</code>	Show UUID/type metadata for a device.	
<code>tune2fs -l DEVICE</code>	Display ext-family filesystem parameters.	
<code>xfs_info MOUNTPOINT</code>	Display XFS filesystem geometry when applicable.	

18. Filesystem Permissions and Security

Markdown file: `commands/filesystem-permissions-security-commands.md`

Command / Example	Purpose	Notes
<code>chmod 755 file.txt</code>	Set rwx for owner and rx for group/others.	
<code>chown user:group file.txt</code>	Change file owner and group.	
<code>chgrp group file.txt</code>	Change group ownership.	
<code>umask 022</code>	Set a default mask commonly resulting in 755 directories and 644 files.	
<code>setfacl</code>	Set filesystem ACLs.	
<code>getfacl</code>	Read filesystem ACLs.	
<code>chmod 600 PRIVATE_KEY</code>	Allow only the owner to read/write a private key.	
<code>chmod 700 DIRECTORY</code>	Allow only the owner full access to a directory.	
<code>chmod -R g+rX PATH</code>	Recursively grant group read and directory/search permissions without blindly making every file executable.	
<code>chmod 1777 DIRECTORY</code>	Apply world-writable permissions plus sticky bit, like /tmp.	
<code>chmod 2775 DIRECTORY</code>	Apply setgid so new entries inherit the directory group.	
<code>namei -l /path/to/file</code>	Show permissions for every component of a path.	
<code>setfacl -m g:GROUP:rx PATH</code>	Grant a group read/execute ACL permissions.	
<code>setfacl -d -m g:GROUP:rwx DIRECTORY</code>	Set a default ACL inherited by newly created entries.	

Command / Example	Purpose	Notes
<code>getfacl -p PATH</code>	Display ACLs with absolute path information.	

19. Containerization and Orchestration

Markdown file: `commands/containerization-orchestration-commands.md`

Command / Example	Purpose	Notes
<code>docker run IMAGE</code>	Run a container from an image.	
<code>docker ps</code>	List running containers.	
<code>docker ps -a</code>	List all containers, including stopped ones.	
<code>docker build -t IMAGE_NAME .</code>	Build an image from a Dockerfile.	
<code>docker exec -it CONTAINER_ID bash</code>	Open an interactive Bash shell inside a running container.	
<code>docker stop CONTAINER_ID</code>	Stop a container.	
<code>docker rm CONTAINER_ID</code>	Remove a container.	
<code>docker logs CONTAINER_ID</code>	View container logs.	
<code>docker images</code>	List local images.	
<code>docker rmi IMAGE_NAME</code>	Remove an image.	
<code>docker network ls</code>	List Docker networks.	
<code>docker-compose up</code>	Start a multi-container Compose application.	Legacy standalone syntax; modern Docker uses <code>docker compose</code> .
<code>docker-compose down</code>	Stop/remove a Compose application.	Legacy standalone syntax; modern Docker uses <code>docker compose</code> .
<code>docker-compose logs</code>	View Compose logs.	Legacy standalone syntax; modern Docker uses <code>docker compose</code> .
<code>kubectl get pods</code>	List pods in the current namespace.	
<code>kubectl get nodes</code>	List cluster nodes.	
<code>kubectl get services</code>	List services.	
<code>kubectl apply -f FILE.yaml</code>	Apply declarative Kubernetes configuration.	
<code>kubectl create -f FILE.yaml</code>	Create Kubernetes resources from a file.	
<code>kubectl delete -f FILE.yaml</code>	Delete resources described by a file.	
<code>kubectl exec -it POD -- bash</code>	Execute Bash in a pod.	
<code>kubectl logs POD</code>	View pod logs.	

Command / Example	Purpose	Notes
<code>kubectl describe pod POD</code>	Show detailed pod information and events.	
<code>kubectl scale deployment DEPLOYMENT --replicas=N</code>	Scale a deployment.	
<code>kubectl rollout restart deployment DEPLOYMENT</code>	Restart a deployment rollout.	
<code>kubectl port-forward pod/POD LOCAL:REMOTE</code>	Forward a pod port to localhost.	
<code>helm install RELEASE CHART</code>	Install a Helm chart.	
<code>helm upgrade RELEASE CHART</code>	Upgrade a Helm release.	
<code>helm list</code>	List Helm releases.	
<code>helm delete RELEASE</code>	Delete a Helm release.	helm uninstall is the preferred modern spelling.
<code>helm search repo TERM</code>	Search configured Helm repositories.	
<code>docker pull IMAGE</code>	Pull an image from a registry.	
<code>docker inspect OBJECT</code>	Show detailed JSON metadata for a Docker object.	
<code>docker stats</code>	Stream container CPU, memory, network and I/O usage.	
<code>docker system df</code>	Show Docker disk usage.	
<code>docker system prune</code>	Remove unused Docker objects.	Review what will be removed before confirming.
<code>docker volume ls</code>	List Docker volumes.	
<code>docker compose up -d</code>	Start a Compose application in detached mode.	
<code>docker compose ps</code>	Show Compose application containers.	
<code>docker compose logs -f</code>	Follow Compose logs.	
<code>docker compose down -v</code>	Stop Compose services and remove volumes.	Removing volumes can delete persisted application data.
<code>kubectl get pods -A -o wide</code>	List pods across namespaces with node/IP information.	
<code>kubectl config get-contexts</code>	List kubeconfig contexts.	
<code>kubectl config use-context CONTEXT</code>	Switch the active Kubernetes context.	
<code>kubectl get events --sort-by=.metadata.creationTimestamp</code>	List events chronologically.	
<code>kubectl logs -f POD</code>	Follow pod logs.	
<code>kubectl logs POD --previous</code>	Show logs from the previous terminated container instance.	

Command / Example	Purpose	Notes
<code>kubectl exec -it POD -- /bin/sh</code>	Open a POSIX shell in a pod when Bash is unavailable.	
<code>kubectl rollout status deployment/DEPLOYMENT</code>	Wait for and report rollout status.	
<code>kubectl rollout undo deployment/DEPLOYMENT</code>	Roll back a deployment to the previous revision.	
<code>kubectl top pods</code>	Show pod CPU/memory metrics when metrics-server is available.	
<code>kubectl diff -f FILE.yaml</code>	Preview declarative changes before applying.	
<code>kubectl explain RESOURCE</code>	Show schema/help for a Kubernetes resource.	
<code>helm repo add NAME URL</code>	Add a Helm chart repository.	
<code>helm repo update</code>	Refresh chart repository indexes.	
<code>helm upgrade --install RELEASE CHART</code>	Install or upgrade a release idempotently.	
<code>helm status RELEASE</code>	Show release status.	
<code>helm history RELEASE</code>	Show release revisions.	
<code>helm rollback RELEASE REVISION</code>	Roll back to a prior revision.	
<code>helm uninstall RELEASE</code>	Uninstall a release.	
<code>helm template RELEASE CHART</code>	Render chart templates locally without installing.	

20. Automation and Configuration Management

Markdown file: [commands/automation-configuration-management-commands.md](#)

Command / Example	Purpose	Notes
<code>ansible all -m ping</code>	Ping all inventory hosts using Ansible.	
<code>ansible-playbook playbook.yml</code>	Run an Ansible playbook.	
<code>ansible HOST -m command -a 'COMMAND'</code>	Run an ad-hoc command on a target host.	
<code>ansible-playbook --check playbook.yml</code>	Run a playbook in check/dry-run mode where supported.	
<code>ansible-playbook --limit HOST playbook.yml</code>	Limit a playbook run to a host/group.	
<code>ansible-playbook --extra-vars "key=value" playbook.yml</code>	Pass extra variables.	
<code>terraform init</code>	Initialize a Terraform working directory.	
<code>terraform plan</code>	Preview proposed infrastructure changes.	
<code>terraform apply</code>	Apply Terraform changes.	

Command / Example	Purpose	Notes
terraform destroy	Destroy Terraform-managed infrastructure.	Destructive; review plan and target carefully.
terraform validate	Validate Terraform configuration.	
terraform show	Show Terraform state or a saved plan.	
puppet apply manifest.pp	Apply a Puppet manifest locally.	
puppet agent --test	Run a one-off Puppet agent cycle.	
puppet resource	Show or manage Puppet resource state.	
ansible-inventory --graph	Display inventory hierarchy.	
ansible-playbook --syntax-check playbook.yml	Validate playbook syntax.	
ansible-playbook --diff playbook.yml	Show file/template differences produced by a playbook.	
ansible-playbook --tags TAG playbook.yml	Run only tasks matching a tag.	
ansible-vault encrypt FILE	Encrypt a sensitive Ansible data file.	
ansible-vault view FILE	View an encrypted Ansible Vault file.	
terraform fmt -recursive	Format Terraform files recursively.	
terraform providers	Show provider requirements.	
terraform state list	List resources recorded in Terraform state.	
terraform output	Display Terraform output values.	
terraform workspace list	List workspaces.	
terraform import ADDRESS ID	Import an existing object into Terraform state.	
puppet parser validate manifest.pp	Validate Puppet manifest syntax.	
puppet apply --noop manifest.pp	Preview Puppet changes without enforcing them.	

21. CI/CD Tools and Commands

Markdown file: `commands/cicd-tools-commands.md`

Command / Example	Purpose	Notes
java -jar jenkins.war	Start Jenkins from a WAR file.	
http://localhost:8080	Default local Jenkins web endpoint in many standalone setups.	URL, not a shell command.
.gitlab-ci.yml	GitLab CI/CD pipeline configuration file.	Configuration file, not a shell command.
gitlab-runner register	Register a GitLab Runner.	
gitlab-runner run	Run GitLab Runner to process jobs.	

Command / Example	Purpose	Notes
<code>actions/checkout@v2</code>	Source screenshot example of the GitHub checkout action.	Older major shown in the source; use a currently maintained action version in real workflows.
<code>actions/setup-node@v2</code>	Source screenshot example of the Node setup action.	Older major shown in the source.
<code>docker/setup-buildx-action@v1</code>	Source screenshot example for Docker Buildx setup.	Older major shown in the source.
<code>gitlab-runner list</code>	List configured GitLab runners.	
<code>gitlab-runner verify</code>	Verify registered runners can connect to GitLab.	
<code>gitlab-runner status</code>	Show GitLab Runner service status where supported.	
<code>gh workflow list</code>	List GitHub Actions workflows using GitHub CLI.	
<code>gh workflow run WORKFLOW</code>	Manually dispatch a GitHub Actions workflow.	
<code>gh run list</code>	List recent GitHub Actions runs.	
<code>gh run view RUN_ID --log</code>	View logs for a GitHub Actions run.	
<code>gh run rerun RUN_ID --failed</code>	Rerun failed jobs in a workflow run.	
<code>curl -fsS http://localhost:8080/login</code>	Quickly test whether a local Jenkins HTTP endpoint is responding.	

22. Cloud Services

Markdown file: `commands/cloud-services-cli-commands.md`

Command / Example	Purpose	Notes
<code>aws configure</code>	Configure AWS CLI credentials/defaults.	
<code>aws s3 cp file.txt s3://bucket-name/</code>	Copy a file to an S3 bucket.	
<code>aws ec2 describe-instances</code>	Describe EC2 instances.	
<code>aws ec2 start-instances --instance-ids ID</code>	Start an EC2 instance.	
<code>aws ec2 stop-instances --instance-ids ID</code>	Stop an EC2 instance.	
<code>aws s3 sync LOCAL/ s3://BUCKET/PREFIX/</code>	Synchronize a directory to S3.	
<code>az login</code>	Sign in with Azure CLI.	
<code>az vm list</code>	List Azure virtual machines.	
<code>az vm start --name VM --resource-group RG</code>	Start an Azure VM.	
<code>az storage blob upload</code>	Upload a blob to Azure Storage.	

Command / Example	Purpose	Notes
<code>az group create --name RG --location REGION</code>	Create an Azure resource group.	
<code>gcloud auth login</code>	Sign in with Google Cloud CLI.	
<code>gcloud compute instances list</code>	List Compute Engine instances.	
<code>gcloud compute instances stop INSTANCE</code>	Stop a Compute Engine instance.	
<code>gcloud app browse</code>	Open the current App Engine application in a browser.	
<code>aws sts get-caller-identity</code>	Show the active AWS identity/account.	
<code>aws configure list</code>	Show where AWS CLI configuration values came from.	
<code>aws configure --profile PROFILE</code>	Configure a named AWS CLI profile.	
<code>aws s3 ls</code>	List S3 buckets or objects.	
<code>aws logs tail LOG_GROUP --follow</code>	Follow CloudWatch Logs output.	
<code>aws eks update-kubeconfig --name CLUSTER --region REGION</code>	Update kubeconfig for an Amazon EKS cluster.	
<code>az account show</code>	Show the active Azure subscription and tenant.	
<code>az account list -o table</code>	List Azure subscriptions.	
<code>az account set --subscription SUBSCRIPTION</code>	Select the active Azure subscription.	
<code>az group list -o table</code>	List resource groups.	
<code>az resource list -o table</code>	List resources in the current subscription.	
<code>az vm list -d -o table</code>	List VMs with power state and public IP information.	
<code>az aks get-credentials -g RG -n CLUSTER</code>	Merge AKS credentials into kubeconfig.	
<code>gcloud auth list</code>	Show authenticated Google Cloud accounts.	
<code>gcloud config list</code>	Show active gcloud configuration.	
<code>gcloud config set project PROJECT_ID</code>	Set the active Google Cloud project.	
<code>gcloud projects list</code>	List accessible projects.	
<code>gcloud compute instances start INSTANCE --zone ZONE</code>	Start a Compute Engine VM.	
<code>gcloud storage ls</code>	List Cloud Storage buckets/objects.	
<code>gcloud container clusters get-credentials CLUSTER --region REGION</code>	Update kubeconfig for a GKE cluster.	

Command / Example	Purpose	Notes
<code>gcloud logging read "resource.type=gce_instance" --limit=20</code>	Read recent Cloud Logging entries matching a filter.	

23. Logging and Monitoring

Markdown file: `commands/logging-monitoring-commands.md`

Command / Example	Purpose	Notes
<code>prometheus</code>	Start the Prometheus server binary.	
<code>prometheus --config.file=CONFIG_FILE</code>	Start Prometheus with a specific configuration file.	
<code>grafana-cli plugins install PLUGIN_NAME</code>	Install a Grafana plugin using the legacy grafana-cli entry point.	
<code>curl -XGET 'localhost:9200/_cluster/health?pretty'</code>	Get Elasticsearch cluster health.	
<code>logstash -f CONFIG_FILE</code>	Run Logstash with a configuration file.	
<code>http://localhost:5601</code>	Typical local Kibana web endpoint.	URL, not a shell command.
<code>promtool check config prometheus.yml</code>	Validate Prometheus configuration.	
<code>promtool check rules rules.yml</code>	Validate Prometheus rule files.	
<code>promtool query instant http://localhost:9090 'up'</code>	Run an instant PromQL query from the CLI.	
<code>curl -fsS http://localhost:9090/-/healthy</code>	Check Prometheus health endpoint.	
<code>grafana-cli plugins ls</code>	List installed Grafana plugins on installations that provide grafana-cli.	
<code>curl -s 'localhost:9200/_cat/indices?v'</code>	List Elasticsearch indices.	
<code>curl -s 'localhost:9200/_cat/nodes?v'</code>	List Elasticsearch nodes.	
<code>curl -s -H 'Content-Type: application/json' localhost:9200/INDEX/_search -d '{"query":{"match_all":{}}}'</code>	Run a basic Elasticsearch search request.	
<code>logstash -t -f CONFIG_FILE</code>	Test Logstash configuration syntax.	
<code>journalctl -f</code>	Follow the systemd journal.	

Command / Example	Purpose	Notes
<code>journalctl -u SERVICE --since today</code>	Show service logs since the start of today.	
<code>tail -F /var/log/FILE</code>	Follow a log file across rotations/replacements.	

24. Git Version Control

Markdown file: [commands/git-version-control-commands.md](#)

Command / Example	Purpose	Notes
<code>git init</code>	Initialize a Git repository.	
<code>git clone URL</code>	Clone a repository.	
<code>git status</code>	Show working-tree and staging status.	
<code>git add FILE</code>	Stage changes.	
<code>git add -p</code>	Interactively stage selected hunks.	
<code>git commit -m "MESSAGE"</code>	Commit staged changes.	
<code>git log --oneline --graph --decorate --all</code>	Show compact decorated history graph.	
<code>git diff</code>	Show unstaged changes.	
<code>git diff --staged</code>	Show staged changes.	
<code>git branch</code>	List local branches.	
<code>git switch BRANCH</code>	Switch branches.	
<code>git switch -c NEW_BRANCH</code>	Create and switch to a branch.	
<code>git merge BRANCH</code>	Merge a branch into the current branch.	
<code>git rebase BASE</code>	Replay commits on a new base.	
<code>git fetch --all --prune</code>	Fetch all remotes and remove stale remote-tracking refs.	
<code>git pull --rebase</code>	Fetch and replay local commits on top of the remote branch.	
<code>git push -u origin BRANCH</code>	Push a branch and set its upstream.	
<code>git remote -v</code>	List remotes and URLs.	
<code>git stash push -m "MESSAGE"</code>	Temporarily save uncommitted changes.	
<code>git stash list</code>	List stashes.	
<code>git stash pop</code>	Apply and remove the latest stash.	
<code>git restore FILE</code>	Restore working-tree content from the index/HEAD.	
<code>git restore --staged FILE</code>	Unstage a file without discarding working-tree changes.	

Command / Example	Purpose	Notes
<code>git revert COMMIT</code>	Create a new commit that reverses a prior commit.	
<code>git reset --soft HEAD~1</code>	Move HEAD back one commit while keeping changes staged.	
<code>git cherry-pick COMMIT</code>	Apply a specific commit onto the current branch.	
<code>git tag -a VERSION -m "MESSAGE"</code>	Create an annotated tag.	
<code>git reflog</code>	Show local reference movement history; useful for recovery.	
<code>git bisect start</code>	Start binary search for a regression.	
<code>git clean -nfd</code>	Preview removal of untracked files/directories.	
<code>git clean -fd</code>	Remove untracked files/directories.	Destructive; run the -n preview first.

25. Shell and Environment

Markdown file: `commands/shell-environment-commands.md`

Command / Example	Purpose	Notes
<code>echo "\$VAR"</code>	Print a shell variable.	
<code>env</code>	Print the process environment.	
<code>printenv VAR</code>	Print one environment variable.	
<code>export VAR=value</code>	Set and export an environment variable to child processes.	
<code>unset VAR</code>	Remove a shell variable/environment variable.	
<code>source FILE</code>	Execute shell code from a file in the current shell.	
<code>. FILE</code>	POSIX shorthand for source.	
<code>which COMMAND</code>	Locate an executable via PATH (implementation varies).	
<code>command -v COMMAND</code>	Portable way to determine how a command resolves.	
<code>type COMMAND</code>	Show whether a name is an alias, builtin, function or executable.	
<code>alias ll='ls -lah'</code>	Create a shell alias.	
<code>unalias NAME</code>	Remove an alias.	
<code>history</code>	Show shell command history.	
<code>history grep PATTERN</code>	Search shell history.	
<code>COMMAND > file.log 2>&1</code>	Redirect standard output and error to a file.	
<code>COMMAND 2> errors.log</code>	Redirect standard error only.	

Command / Example	Purpose	Notes
COMMAND1 COMMAND2	Pipe one command output into another.	
COMMAND1 && COMMAND2	Run the second command only if the first succeeds.	
COMMAND1 COMMAND2	Run the second command only if the first fails.	
set -euo pipefail	Common Bash strict-mode settings for scripts.	Understand each option before adopting it in existing scripts.
trap "COMMAND" EXIT	Run cleanup logic when a shell script exits.	
read -r VAR	Read a line from standard input without backslash interpretation.	

26. JSON, YAML and Structured Data

Markdown file: [commands/json-yaml-structured-data-commands.md](#)

Command / Example	Purpose	Notes
jq '.' file.json	Pretty-print and validate JSON with jq.	
jq -r '.name' file.json	Extract a string value without JSON quotes.	
jq '.items[] {name: .metadata.name}' file.json	Transform JSON arrays/objects.	
curl -s URL jq '.'	Pretty-print JSON returned by an HTTP endpoint.	
python3 -m json.tool file.json	Pretty-print and validate JSON using Python standard library.	
yq '.metadata.name' file.yaml	Read a YAML field with yq (syntax depends on yq implementation/version).	
yq -o=json '.' file.yaml	Convert YAML to JSON with Mike Farah yq.	
base64 FILE	Encode a file as Base64.	
base64 -d FILE.b64	Decode Base64 on GNU/Linux.	
xmllint --format file.xml	Pretty-print XML (libxml2 tools).	
xmllint --xpath '//name/text()' file.xml	Extract XML data with XPath.	
column -s, -t file.csv	Render simple CSV-like data as aligned columns.	

27. Security, Cryptography and Certificates

Markdown file: [commands/security-crypto-certificates-commands.md](#)

Command / Example	Purpose	Notes
<code>sha256sum FILE</code>	Calculate a SHA-256 checksum on Linux.	
<code>shasum -a 256 FILE</code>	Calculate a SHA-256 checksum on macOS and other systems with shasum.	
<code>md5sum FILE</code>	Calculate an MD5 checksum for non-security integrity comparisons.	MD5 is cryptographically broken; do not use it for security assurances.
<code>openssl version -a</code>	Show OpenSSL version and build details.	
<code>openssl s_client -connect HOST:443 -servername HOST</code>	Inspect a TLS handshake, certificate chain and negotiated parameters.	
<code>openssl x509 -in cert.pem -noout -subject -issuer -dates</code>	Display certificate subject, issuer and validity dates.	
<code>openssl x509 -in cert.pem -text -noout</code>	Display full X.509 certificate details.	
<code>openssl req -new -newkey rsa:2048 -nodes -keyout key.pem -out request.csr</code>	Generate an RSA private key and CSR.	Protect the generated private key.
<code>openssl rand -hex 32</code>	Generate 32 random bytes encoded as hexadecimal.	
<code>ssh-keygen -t ed25519 -C "COMMENT"</code>	Generate an Ed25519 SSH key pair.	
<code>ssh-keygen -lf ~/.ssh/id_ed25519.pub</code>	Show an SSH public-key fingerprint.	
<code>ssh-keyscan -H HOST >> ~/.ssh/known_hosts</code>	Fetch and append host keys.	Verify fingerprints through a trusted channel before relying on them.
<code>gpg --list-keys</code>	List GPG public keys.	
<code>gpg --verify SIGNATURE FILE</code>	Verify a detached GPG signature.	
<code>getcap FILE</code>	Show Linux file capabilities.	

28. Database CLI

Markdown file: [commands/database-cli-commands.md](#)

Command / Example	Purpose	Notes
-------------------	---------	-------

Command / Example	Purpose	Notes
<code>psql -h HOST -U USER -d DATABASE</code>	Connect to PostgreSQL.	
<code>psql -c "SELECT version();" </code>	Run a PostgreSQL query from the shell.	
<code>psql -f script.sql</code>	Execute SQL from a file.	
<code>\l</code>	List PostgreSQL databases inside psql.	
<code>\dt</code>	List tables inside psql.	
<code>\d TABLE</code>	Describe a PostgreSQL table inside psql.	
<code>\q</code>	Quit psql.	
<code>mysql -h HOST -u USER -p</code>	Connect to MySQL/MariaDB and prompt for a password.	
<code>mysql -u USER -p DATABASE < script.sql</code>	Execute a SQL script in MySQL/MariaDB.	
<code>SHOW DATABASES;</code>	List databases inside MySQL/MariaDB.	
<code>SHOW TABLES;</code>	List tables in the active database.	
<code>DESCRIBE TABLE;</code>	Describe a table.	
<code>sqlite3 database.db</code>	Open a SQLite database.	
<code>.tables</code>	List tables in sqlite3.	
<code>.schema TABLE</code>	Show a table schema in sqlite3.	
<code>.headers on</code>	Show column headers in sqlite3 output.	
<code>.mode column</code>	Use aligned column output in sqlite3.	
<code>.quit</code>	Quit sqlite3.	
<code>redis-cli ping</code>	Test a Redis server connection.	
<code>redis-cli INFO</code>	Show Redis server information.	
<code>redis-cli --scan</code>	Iterate keys using SCAN rather than blocking KEYS *.	

29. GitHub CLI

Markdown file: `commands/github-cli-commands.md`

Command / Example	Purpose	Notes
<code>gh auth login</code>	Authenticate GitHub CLI.	
<code>gh auth status</code>	Show authentication status.	
<code>gh repo clone OWNER/REPO</code>	Clone a GitHub repository.	
<code>gh repo view --web</code>	Open the current repository in a browser.	
<code>gh repo create</code>	Create a GitHub repository interactively or with flags.	
<code>gh issue list</code>	List issues.	
<code>gh issue create</code>	Create an issue.	
<code>gh pr list</code>	List pull requests.	

Command / Example	Purpose	Notes
gh pr checkout NUMBER	Check out a pull request locally.	
gh pr create	Create a pull request.	
gh pr checks NUMBER	Show pull-request checks.	
gh pr merge NUMBER	Merge a pull request subject to repository policy.	
gh workflow list	List workflows.	
gh workflow run WORKFLOW	Dispatch a workflow.	
gh run list	List workflow runs.	
gh run watch RUN_ID	Watch a workflow run.	
gh run view RUN_ID --log	View workflow-run logs.	
gh release list	List releases.	
gh release create TAG FILES...	Create a GitHub release and upload assets.	
gh api repos/{owner}/{repo}	Call the GitHub API with authentication.	

30. Python and Virtual Environments

Markdown file: commands/python-virtual-environment-commands.md

Command / Example	Purpose	Notes
python3 --version	Show the Python 3 version.	
python3 -m venv .venv	Create a virtual environment.	
source .venv/bin/activate	Activate a virtual environment on POSIX shells.	
deactivate	Leave the active Python virtual environment.	
python -m pip install PACKAGE	Install a Python package using the interpreter-associated pip.	
python -m pip install -r requirements.txt	Install requirements from a file.	
python -m pip list	List installed packages.	
python -m pip freeze > requirements.txt	Write exact installed package versions to a requirements file.	
python -m pip check	Check installed package dependency consistency.	
python -m pytest	Run tests with pytest when installed.	
python -m http.server 8000	Start a simple local HTTP file server.	
python -m json.tool file.json	Pretty-print/validate JSON.	

31. Node.js and npm

Markdown file: commands/nodejs-npm-commands.md

Command / Example	Purpose	Notes
<code>node --version</code>	Show the Node.js version.	
<code>npm --version</code>	Show the npm version.	
<code>npm init -y</code>	Create a default package.json.	
<code>npm install</code>	Install dependencies from package.json.	
<code>npm install PACKAGE</code>	Add a package dependency.	
<code>npm install -D PACKAGE</code>	Add a development dependency.	
<code>npm ci</code>	Install dependencies exactly from package-lock.json; useful in CI.	
<code>npm run SCRIPT</code>	Run a package.json script.	
<code>npm test</code>	Run the configured test script.	
<code>npm outdated</code>	Show outdated packages.	
<code>npm audit</code>	Check dependencies against npm security advisories.	
<code>npm audit fix</code>	Apply compatible dependency updates for known vulnerabilities.	Review resulting dependency changes and retest.
<code>npm COMMAND</code>	Run a package-provided command without a permanent global install in common workflows.	

32. Anthropic Claude / Claude Code

Markdown file: commands/anthropic-claude-commands.md

Command / Example	Purpose	Notes
<code>claude</code>	Start an interactive Claude Code session.	Run from the project directory you want Claude to work with.
<code>claude "explain this project"</code>	Start an interactive session with an initial prompt.	
<code>claude -p "explain this function"</code>	Run a non-interactive prompt and exit.	Useful for scripts and CI-style workflows.
<code>cat logs.txt claude -p "explain these errors"</code>	Pipe content to Claude Code for analysis.	Review data sensitivity before piping logs or files.
<code>claude -c</code>	Continue the most recent conversation for the current directory.	

Command / Example	Purpose	Notes
<code>claude -r "<session>"</code> <code>"continue this task"</code>	Resume a session by ID or name.	
<code>claude auth login</code>	Sign in to Anthropic.	
<code>claude auth status --text</code>	Show current authentication status.	Useful as a pre-flight check.
<code>claude auth logout</code>	Sign out of Anthropic.	
<code>claude doctor</code>	Run read-only installation and settings diagnostics.	A good first troubleshooting command.
<code>claude update</code>	Update Claude Code to the latest version.	Blocked if <code>DISABLE_UPDATES</code> is enforced; auto-update can be disabled separately.
<code>claude --model sonnet</code>	Start a session with a selected model alias.	A full supported model ID may also be used.
<code>claude -p --max-turns 3 "query"</code>	Limit agentic turns for one non-interactive run.	Helps bound execution and cost.
<code>claude --permission-mode plan</code>	Start in plan permission mode.	Prefer safer permission modes when reviewing unfamiliar repositories.
<code>claude --debug-file /tmp/claude-debug.log</code>	Enable debugging and write logs to a specified file.	Debug logs may contain operational details; handle them appropriately.
<code>claude mcp</code>	Configure Model Context Protocol (MCP) servers.	
<code>export ANTHROPIC_API_KEY="<your-api-key>"</code>	Set an Anthropic API key for the current shell.	Secret: do not commit or expose in shell history or screenshots.
<code>test -n "\$ANTHROPIC_API_KEY" && echo "ANTHROPIC_API_KEY is set"</code>	Check whether the API key variable is set without printing it.	Safer than echoing the secret value.
<code>unset ANTHROPIC_API_KEY</code>	Remove the Anthropic API key from the current shell.	
<code>export ANTHROPIC_AUTH_TOKEN="<your-bearer-token>"</code>	Set a custom Authorization bearer-token value.	Secret: protect it as you would an API key.
<code>export ANTHROPIC_BASE_URL="https://gateway.example.com/anthropic"</code>	Route API requests through a proxy, gateway, or custom Anthropic-compatible endpoint.	Validate gateway compatibility before production use.
<code>export CLAUDE_CODE_USE_BEDROCK=1</code>	Route Claude Code through Amazon Bedrock.	Provider credentials and permissions must already be configured.

Command / Example	Purpose	Notes
export ANTHROPIC_BEDROCK_BASE_URL="https://bedrock-gateway.example.com"	Override the Amazon Bedrock endpoint.	Useful for custom endpoints or an LLM gateway.
export CLAUDE_CODE_USE_VERTEX=1	Route Claude Code through the Google Cloud provider path.	Provider credentials and project configuration are required.
export ANTHROPIC_VERTEX_PROJECT_ID="<gcp-project-id>"	Set the GCP project ID for provider requests.	
export HTTPS_PROXY="http://proxy.example.com:8080"	Configure an HTTPS proxy.	Do not embed reusable credentials in a committed script.
export HTTP_PROXY="http://proxy.example.com:8080"	Configure an HTTP proxy.	
export NO_PROXY="localhost,127.0.0.1,.example.internal"	Bypass the proxy for selected domains and IP addresses.	
export NODE_EXTRA_CA_CERTS="/path/to/corporate-ca.pem"	Add a custom CA certificate for Node.js TLS connections.	Useful on corporate TLS-inspection networks.
export CLAUDE_CODE_CERT_STORE="bundled,system"	Select Claude Code CA certificate sources.	Current documented default is bundled,system on supported runtimes.
export CLAUDE_CODE_CLIENT_CERT="/path/to/client-cert.pem"	Set the client certificate path for mTLS.	Pair with CLAUDE_CODE_CLIENT_KEY.
export CLAUDE_CODE_CLIENT_KEY="/path/to/client-key.pem"	Set the client private-key path for mTLS.	Secret material: restrict file permissions.
chmod 600 /path/to/client-key.pem	Restrict the mTLS private key to the file owner.	Verify ownership as well as permissions.
export DISABLE_TELEMETRY=1	Opt out of Claude Code telemetry.	Any non-empty value enables this behavior; unset it to restore the default.
export CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC=1	Disable non-essential network traffic.	Any non-empty value enables this behavior; can affect features that need flag fetching.
export DISABLE_AUTOUPDATER=1	Disable automatic background updates.	Manual claude update remains available.
export CLAUDE_CODE_MAX_TURNS=10	Set a default cap for agentic turns.	An explicit --max-turns flag takes precedence.

Command / Example	Purpose	Notes
env cut -d= -f1 grep -E '^(ANTHROPIC CLAUDE_CODE DISABLE_ NODE_EXTRA_CA_ CERTS HTTPS_PROXY HTTP_P ROXY NO_PROXY)\$'	List Claude-related environment variable names without printing values.	Safer for troubleshooting than dumping secrets.

References: code.claude.com/docs/en/cli-reference and code.claude.com/docs/en/env-vars

Disclaimer | Warning!

- *Confirm the target host, cloud account, subscription/project and Kubernetes context before making changes.*
- *Use dry-run, plan, diff, preview or read-only commands before destructive operations whenever the tool supports them.*
- *Do not place passwords, tokens, private keys or production secrets directly in commands that will be stored in shell history.*
- *Capture logs and current state before troubleshooting changes so you can compare before/after behavior.*
- *Use `COMMAND --help`, `man COMMAND` or the **official tool documentation** when flags differ by operating system or version.*
- *Test high-impact commands in a **non-production environment and maintain verified backups** for data-changing operations.*

Note: Imoh Etuk is not responsible for destructive commands run without proper testing that breaks your production systems.